

25) Prolog Program to Implement Monkey Banana Problem

Aim

To write a Prolog program to solve the Monkey Banana Problem using logical reasoning and state-space search.

Objectives

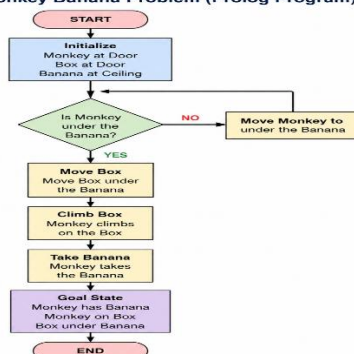
- To represent the positions of the monkey, box, and bananas.
- To move the monkey and box.
- To allow the monkey to climb the box.
- To obtain the bananas.

Algorithm

1. Start with the initial state of the monkey, box, and bananas.
2. Check whether the monkey is under the bananas.
3. If not, move the monkey to the required position.
4. Push the box below the bananas.
5. Climb onto the box.
6. Take the bananas.
7. Display the solution.

Flow Chart

Flowchart – Monkey Banana Problem (Prolog Program)



Prolog Code

% Monkey Banana Problem

```
move(state(monkey(Pos), box(Pos), bananas(ceiling)),
     state(monkey(Pos), box(Pos), bananas(ceiling))) :-
    Pos = under_bananas.

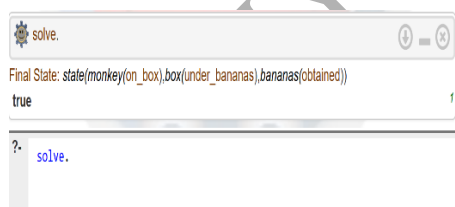
move(state(monkey(Pos1), box(Pos1), bananas(ceiling)),
     state(monkey(under_bananas), box(under_bananas), bananas(ceiling))) :-
    Pos1 = door.

climb(state(monkey(under_bananas), box(under_bananas), bananas(ceiling)),
      state(monkey(on_box), box(under_bananas), bananas(ceiling))).

take_banana(state(monkey(on_box), box(under_bananas), bananas(ceiling)),
            state(monkey(on_box), box(under_bananas), bananas(obtained))).

solve :-
    Initial = state(monkey(door), box(door), bananas(ceiling)),
    move(Initial, State1),
    climb(State1, State2),
    take_banana(State2, Final),
    write('Final State: '),
    write(Final).
```

Sample Output



Result

Thus, the Prolog program successfully solves the Monkey Banana Problem

Conclusion

The program demonstrates how Prolog can be used for state-space problem solving and logical reasoning.

26) Prolog Program for Fruit and Its Color Using Backtracking

Aim

To write a Prolog program to identify the color of fruits using backtracking.

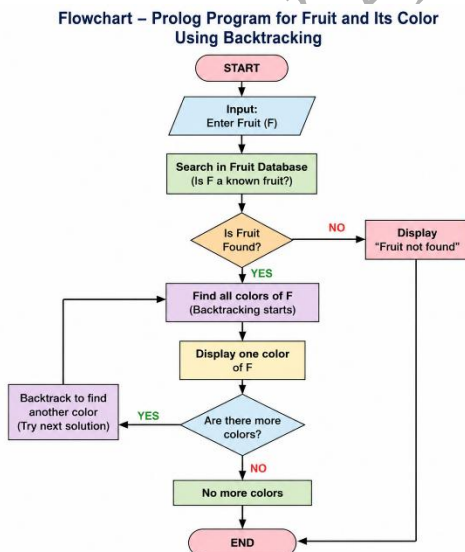
Objectives

- To store fruit and color relationships as facts.
- To use rules for finding fruit colors.
- To demonstrate Prolog backtracking.

Algorithm

1. Define facts representing fruits and their colors.
2. Define a predicate fruit_color/2.
3. Give a fruit as input.
4. Prolog searches for its color.
5. If multiple answers are available, Prolog uses backtracking.
6. Display the result.

Flow Chart



Prolog Program

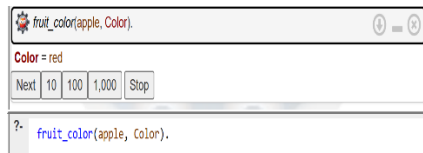
% Fruit and Color using Backtracking

```
fruit(apple).  
fruit(banana).  
fruit(orange).  
fruit(grape).  
fruit(mango).
```

```
color(apple, red).  
color(apple, green).  
color(banana, yellow).  
color(orange, orange).  
color(grape, purple).  
color(grape, green).  
color(mango, yellow).
```

```
fruit_color(Fruit, Color) :-  
    fruit(Fruit),  
    color(Fruit, Color).
```

Sample Output



Result

Thus, the Prolog program successfully finds the color of a fruit using backtracking.

Conclusion

The program demonstrates the use of **facts, predicates, and backtracking** in Prolog.

27) Prolog Program to Implement Best First Search Algorithm

Aim

To write a Prolog program to implement the Best First Search algorithm for finding a path from a start node to a goal node.

Objectives

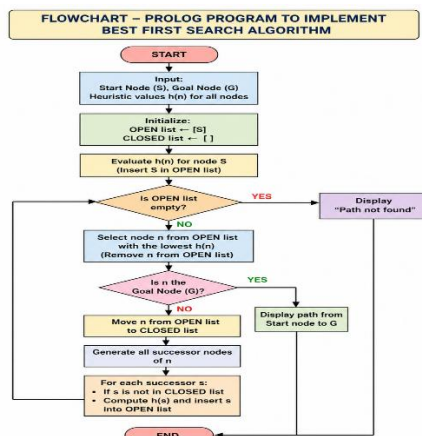
- To represent a graph using Prolog facts.
- To assign heuristic values to nodes.

- To select the node with the lowest heuristic value.
- To find a path from the start node to the goal node.

Algorithm

1. Define the graph using edges.
2. Assign heuristic values to each node.
3. Start from the initial node.
4. Generate its neighboring nodes.
5. Select the node with the smallest heuristic value.
6. Continue searching until the goal node is reached.
7. Display the path.

Flow Chart



Prolog Code

% Best First Search

edge(a, b).

edge(a, c).

edge(b, d).

edge(b, e).

edge(c, f).

edge(c, g).

edge(d, h).

edge(e, h).

edge(f, h).

edge(g, h).

heuristic(a, 7).

heuristic(b, 5).

heuristic(c, 4).

```
heuristic(d, 3).
heuristic(e, 2).
heuristic(f, 2).
heuristic(g, 1).
heuristic(h, 0).
```

```
best_first(Start, Goal, Path) :-
    search([Start], Goal, Path).
```

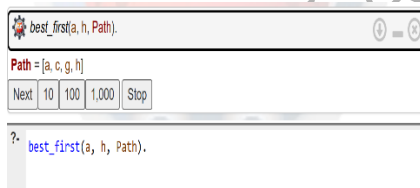
```
search([Goal|_], Goal, [Goal]).
```

```
search([Current|Rest], Goal, [Current|Path]) :-
    findall(H-Next,
        (edge(Current, Next), heuristic(Next, H)),
        Children),
    sort(Children, Sorted),
    extract_nodes(Sorted, Nodes),
    append(Nodes, Rest, NewQueue),
    search(NewQueue, Goal, Path).
```

```
extract_nodes([], []).
```

```
extract_nodes([_Node|T], [Node|Rest]) :-
    extract_nodes(T, Rest).
```

Sample Output



Result

Thus, the Prolog program successfully implements the Best First Search algorithm.

Conclusion

Best First Search selects the most promising node based on its heuristic value and searches toward the goal efficiently.

28) Prolog Program for Medical Diagnosis

Aim

To write a Prolog program for medical diagnosis based on symptoms provided by the user.

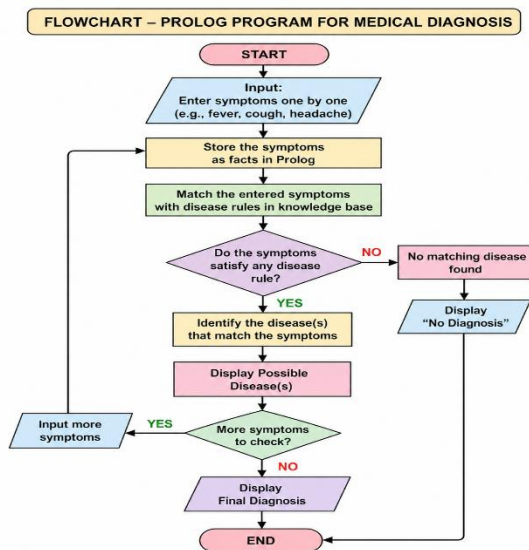
Objectives

- To represent symptoms as facts.
- To define rules for different diseases.
- To accept symptoms as input.
- To identify possible diseases using logical inference.

Algorithm

1. Define disease rules based on symptoms.
2. Take symptoms from the user.
3. Compare the symptoms with disease rules.
4. If the required symptoms match, identify the disease.
5. Display the diagnosis.
6. If no disease matches, display an appropriate message.

FlowChart



Prolog Code

% Medical Diagnosis System

```
symptom(fever).
symptom(cough).
symptom(headache).
symptom(body_pain).
symptom(sore_throat).
```

```
symptom(runny_nose).  
symptom(stomach_pain).  
symptom(vomiting).
```

disease(flu) :-

```
    symptom(fever),  
    symptom(cough),  
    symptom(body_pain).
```

disease(common_cold) :-

```
    symptom(cough),  
    symptom(runny_nose),  
    symptom(sore_throat).
```

disease(migraine) :-

```
    symptom(headache),  
    symptom(vomiting).
```

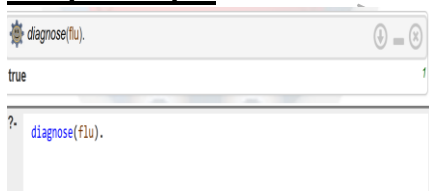
disease(food_poisoning) :-

```
    symptom(stomach_pain),  
    symptom(vomiting).
```

diagnose(Disease) :-

```
    disease(Disease).
```

Sample Output



Result

Thus, the Prolog program successfully performs **basic medical diagnosis using symptoms and logical rules.**

Conclusion

The program demonstrates how Prolog can be used in **expert systems** to make decisions based on predefined medical knowledge and symptoms.